

آموزش نصب پایتون در ویندوز

برای نصب پایتون روی ویندوز، مراحل زیر را دنبال کنید:

مرحله 1: دانلود پایتون

- به وبسایت رسمی پایتون بروید: www.python.org
- از منوی اصلی، گزینه **Downloads** را انتخاب کنید.
- نسخه **ویندوز** مناسب سیستم خود (32 بیتی یا 64 بیتی) را دانلود کنید. معمولاً برای سیستم‌های جدیدتر، نسخه 64 بیتی مناسب است.

مرحله 2: اجرای فایل نصبی

- فایل نصبی دانلودشده (معمولاً با نامی شبیه `python-3.x.x.exe`) را باز کنید.
- در پنجره نصب، گزینه **Add Python to PATH** را حتماً تیک بزنید (این کار بسیار مهم است و دسترسی به پایتون از خط فرمان را آسان می‌کند).
- روی دکمه **Customize Installation** یا **Install Now** را کلیک کنید.
- **Install Now:** پایتون با تنظیمات پیش‌فرض نصب می‌شود.
- **Customize Installation:** امکان انتخاب ویژگی‌ها و مسیر نصب به شما داده می‌شود.

مرحله 3: اتمام نصب

- پس از اتمام فرآیند نصب، پنجره‌ای نمایش داده می‌شود که گزینه **Disable path length limit** را ارائه می‌دهد. روی آن کلیک کنید (اختیاری، اما پیشنهاد می‌شود).
- روی دکمه **Close** کلیک کنید.

مرحله 4: بررسی نصب

- **اجرای Command Prompt (CMD):** کلیدهای **Windows + R** را فشار دهید، سپس `cmd` را تایپ کرده و **Enter** بزنید.
- دستور زیر را اجرا کنید:

```
python --version
```

اگر نسخه پایتون (مثل Python 3.x.x) نمایش داده شد، نصب به درستی انجام شده است.

مفهوم IDE (محیط یکپارچه توسعه نرم افزار)

IDE یا Integrated Development Environment، یک نرم افزار کاربردی است که تمام ابزارها و امکانات لازم برای توسعه و کدنویسی نرم افزار را در یک محیط یکپارچه فراهم می کند. هدف اصلی IDE این است که فرآیند برنامه نویسی را ساده تر و کارآمدتر کند.

در دنیای پایتون، چندین IDE محبوب و کارآمد وجود دارد که بر اساس نیازهای برنامه نویسان انتخاب می شوند. در ادامه، IDE های رایج برای پایتون معرفی شده اند:

- PyCharm
- Visual Studio Code (VS Code)
- Jupyter Notebook
- Spyder
- Thonny
- IDLE
- Anaconda (Navigator)
- Sublime Text
- Atom
- Komodo IDE

آموزش نصب VSCode

مراحل نصب (Visual Studio Code (VS Code)

- **دانلود VS Code:** به وبسایت رسمی VS Code بروید. نسخه ویندوز مناسب سیستم خود (64 بیتی یا 32 بیتی) را دانلود کنید.
- **اجرای فایل نصبی:** فایل دانلودشده (VSCodeSetup-x.x.x.exe) را اجرا کنید. با شرایط و ضوابط موافقت کنید (تیک گزینه **I accept the agreement**).
- **انتخاب مسیر نصب:** مسیر پیش فرض یا مسیر دلخواه خود را انتخاب کنید.
- **تنظیمات اضافی:** گزینه های زیر را تیک بزنید (توصیه شده):
 - Add to PATH (recommended).
 - Register Code as an editor for supported file types.
 - Add a shortcut to the desktop.
 - نصب نرم افزار: روی **Install** کلیک کنید و منتظر بمانید تا فرآیند نصب کامل شود.
- **اجرای VS Code:** گزینه **Launch Visual Studio Code** را تیک بزنید و روی **Finish** کلیک کنید.
- **نصب افزونه پایتون:** پس از اجرای VS Code به بخش **Extensions** (آیکون مربع در سمت چپ) بروید و افزونه **Python** را جستجو و نصب کنید.
- **انتخاب مفسر پایتون:** از منوی **Command Palette** (کلید **Ctrl + Shift + P**) گزینه **Python: Select Interpreter** را انتخاب کنید. مسیر مفسر پایتون نصب شده روی سیستم خود را انتخاب کنید.

آشنایی با ساختار پایه پایتون در کدنویسی

خطوط کد (Statements) :

- کدها به صورت خطی نوشته می‌شوند و هر خط یک دستور یا بیان (Statement) است.
- نیازی به علامت نقطه‌ویرگول (;) در انتهای خطوط نیست (مگر اینکه چند دستور در یک خط نوشته شود).

تورفتگی (Indentation) :

- پایتون از تورفتگی (Indentation) برای تعریف بلاک‌های کد استفاده می‌کند (به جای براکت { } در زبان‌هایی مثل C یا Java).
- تمام خطوط مربوط به یک بلاک باید دارای تورفتگی یکسان باشند.
- استفاده نادرست از تورفتگی منجر به خطا می‌شود.

کامنت‌ها (Comments) :

- برای مستندسازی و توضیح کد استفاده می‌شوند و توسط مفسر اجرا نمی‌شوند.
- کامنت تک‌خطی: با # شروع می‌شود.
- کامنت چندخطی: از سه تا نقل قول (''') یا (""") استفاده می‌شود.

متغیرها و انواع داده‌ها:

- نیازی به تعریف نوع متغیر نیست؛ پایتون به صورت پویا نوع داده را تشخیص می‌دهد.
- از نام‌های معنادار برای متغیرها استفاده کنید.

ورودی و خروجی (Input/Output) :

- از تابع input() برای دریافت داده از کاربر و print() برای نمایش خروجی استفاده می‌شود.

کنترل جریان برنامه:

- شامل شرط‌ها (if, elif, else) و حلقه‌ها (for و while)

توابع (Functions) :

- از توابع برای دسته‌بندی کدهای تکراری استفاده می‌شود.

کتابخانه‌ها و ماژول‌ها:

- می‌توانید از ماژول‌های داخلی یا کتابخانه‌های خارجی برای گسترش قابلیت‌ها استفاده کنید.

مفهوم مفسر (Interpreter) و کامپایلر (Compiler)

در دنیای برنامه‌نویسی، مفسر و کامپایلر ابزارهایی هستند که کدهای نوشته‌شده توسط برنامه‌نویس را به زبانی تبدیل می‌کنند که کامپیوتر می‌تواند آن را بفهمد و اجرا کند. این دو ابزار عملکرد مشابهی دارند، اما روش کارشان متفاوت است. اجازه بدهید به صورت ساده و گام به گام توضیح بدهم:

مفسر: (Interpreter)

- مفسر یک ابزار نرم‌افزاری است که کدهای نوشته‌شده در یک زبان برنامه‌نویسی را خط به خط می‌خواند، تجزیه می‌کند و بلافاصله اجرا می‌کند.
- در واقع، مفسر کد را خط به خط اجرا می‌کند و اگر در حین اجرای برنامه خطایی رخ دهد، اجرای کد متوقف می‌شود و خطا را گزارش می‌دهد.
- زبان‌های پایتون، PHP، جاوا اسکریپت، روبی، پرل و شل اسکریپت مفسری هستند.

ویژگی‌های مفسر:

- اجرای خط به خط: کد را خط به خط می‌خواند و اجرا می‌کند.
- خطایابی ساده‌تر: به محض بروز خطا، اجرای کد متوقف می‌شود و خطا را نمایش می‌دهد.
- کندی اجرا: از آنجا که کد خط به خط اجرا می‌شود، اجرای برنامه‌ها ممکن است کمی کندتر باشد.

کامپایلر: (Compiler)

- کامپایلر نیز مانند مفسر، کدهای برنامه‌نویس را به کدی تبدیل می‌کند که کامپیوتر می‌تواند آن را اجرا کند، اما کامپایلر تمام کد را در یک مرحله تجزیه و تحلیل کرده و سپس تمام آن را به یک فایل اجرایی تبدیل می‌کند.
- این به این معنی است که قبل از اجرا، کامپایلر کل کد را بررسی می‌کند و در صورتی که خطایی وجود داشته باشد، تمام خطاها را گزارش می‌دهد و کد اجرا نمی‌شود تا زمانی که خطاها اصلاح شوند.
- زبان‌های C، C++، جاوا، سی شارپ (C#)، Go، Swift و Rust کامپایلری هستند.

ویژگی‌های کامپایلر:

- تجزیه کد به طور یکباره: تمام کد را به یک فایل اجرایی تبدیل می‌کند.
- اجرای سریع‌تر: پس از کامپایل کردن کد، می‌توانید آن را به سرعت اجرا کنید.
- خطایابی دشوارتر: ممکن است چندین خطا در کد وجود داشته باشد که باید پس از کامپایل گزارش شوند.

Python Syntax

```
print('Hello, Rahkar')
```

```
# کامنت نویسی در پایتون
```

```
'''
```

```
نوشتن کامنت چند سطر
```

```
این شیوه کامنت نویسی را docstring می گویند
```

```
'''
```

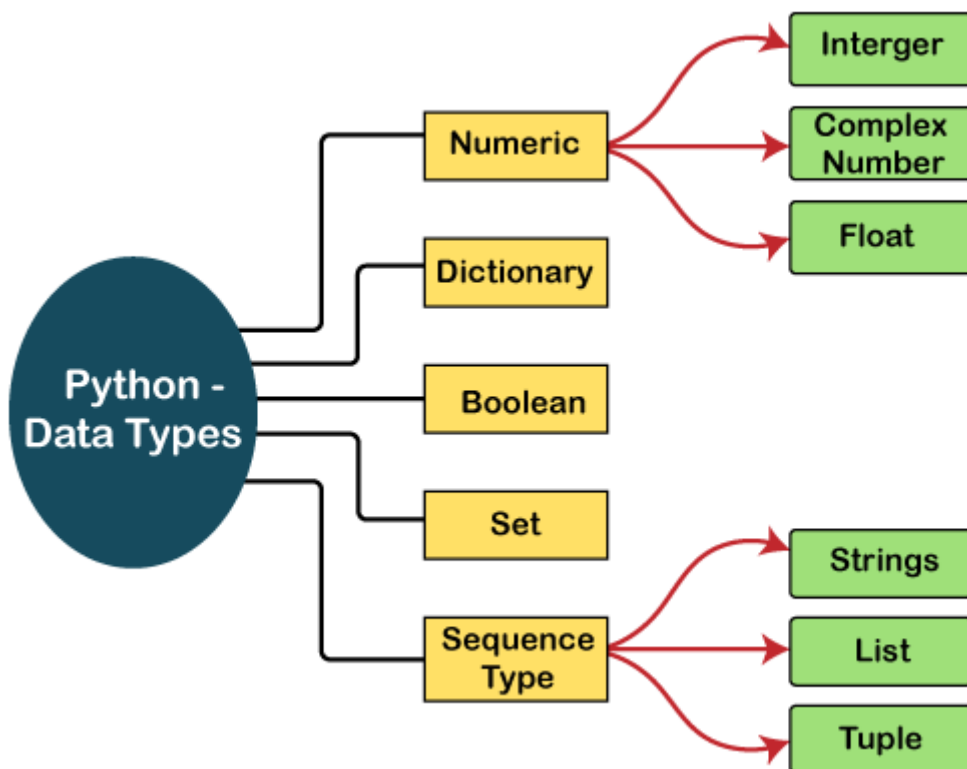
```
print("Hello, Rahkar!")
```

آموزشگاه

کامپیوتر

راهکار

انواع داده در پایتون:



آم

کا

را

عملگرها در پایتون (Operators)

در پایتون، عملگرها ابزارهایی هستند که برای انجام عملیات‌های مختلف روی داده‌ها (مثل اعداد، رشته‌ها، و غیره) استفاده می‌شوند. این عملگرها به دسته‌های مختلفی تقسیم می‌شوند که در اینجا آن‌ها را به تفصیل توضیح خواهیم داد.

1- عملگرهای ریاضی (Arithmetic Operators)

عملگرهای ریاضی برای انجام عملیات‌های پایه‌ای مانند جمع، تفریق، ضرب و تقسیم استفاده می‌شوند.

عملگر	توضیح	مثال	خروجی
+	جمع	$5 + 3$	8
-	تفریق	$5 - 3$	2
*	ضرب	$5 * 3$	15
/	تقسیم (اعشاری)	$5 / 2$	2.5
//	تقسیم صحیح (بدون اعشار)	$5 // 2$	2
%	باقی‌مانده تقسیم (مدول)	$5 \% 2$	1
**	توان	$5 ** 2$	25

2- عملگرهای مقایسه‌ای (Comparison Operators)

عملگرهای مقایسه‌ای برای مقایسه دو مقدار استفاده می‌شوند و نتیجه آن‌ها یک مقدار بولی (True یا False) است.

عملگر	توضیح	مثال	خروجی
==	برابر با	$5 == 3$	False
!=	نابرابر با	$5 != 3$	True
>	بزرگ‌تر از	$5 > 3$	True
<	کوچک‌تر از	$5 < 3$	False
>=	بزرگ‌تر یا مساوی با	$5 >= 3$	True
<=	کوچک‌تر یا مساوی با	$5 <= 3$	False

3- عملگرهای منطقی (Logical Operators)

عملگرهای منطقی برای انجام عملیات‌های منطقی مانند AND, OR و NOT بر روی مقادیر بولی استفاده می‌شوند.

عملگر	توضیح	مثال	خروجی
and	اگر هر دو شرط True باشند، نتیجه True است	True and False	False
or	اگر یکی از شرایط True باشد، نتیجه True است	True or False	True
not	معکوس کردن مقدار بولی	not True	False

4- عملگرهای انتساب (Assignment Operators)

عملگرهای انتساب برای انتساب مقدار به متغیرها استفاده می‌شوند. این عملگرها می‌توانند به شکلی کوتاه‌تر انجام شوند.

عملگر	توضیح	مثال	خروجی
=	انتساب مقدار به متغیر	$x = 5$	$x=5$
+=	افزودن به مقدار متغیر	$x = x + 3$ (معادل $x += 3$)	$x=8$
-=	کم کردن از مقدار متغیر	$x = x - 3$ (معادل $x -= 3$)	$x=2$
*=	ضرب کردن مقدار متغیر با یک عدد	$x = x * 3$ (معادل $x *= 3$)	$x=15$
/=	تقسیم کردن مقدار متغیر با یک عدد	$x = x / 3$ (معادل $x /= 3$)	$x=1.66$
//=	تقسیم صحیح (بدون اعشار)	$x = x // 3$ (معادل $x //= 3$)	$x=1$
%=	محاسبه باقی‌مانده تقسیم	$x = x \% 3$ (معادل $x \% 3$)	$x=2$
**=	محاسبه توان	$x = x ** 2$ (معادل $x ** 2$)	$x=25$

5- عملگرهای عضویت (Membership Operators)

این عملگرها برای بررسی عضویت یک عنصر در یک رشته یا لیست استفاده می‌شوند.

عملگر	توضیح	مثال	خروجی
in	بررسی اینکه آیا یک عنصر در داده‌ها وجود دارد یا خیر	'a' in 'apple'	True
not in	بررسی اینکه آیا یک عنصر در داده‌ها وجود ندارد یا خیر	'b' not in 'apple'	True

6- عملگرهای هویت (Identity Operators)

عملگرهای هویت برای بررسی اینکه آیا دو متغیر به یک شیء اشاره می‌کنند یا نه، استفاده می‌شوند.

عملگر	توضیح	مثال	خروجی
is	بررسی اینکه آیا دو متغیر به یک شیء اشاره می‌کنند یا خیر	x is y	True/False
is not	بررسی اینکه آیا دو متغیر به یک شیء اشاره نمی‌کنند	x is not y	True/False

7- عملگرهای بیت‌وایز (Bitwise Operators)

عملگرهای بیت‌وایز برای انجام عملیات‌های منطقی روی بیت‌های اعداد صحیح استفاده می‌شوند. فعلا وارد این مقوله نمی‌شویم

اولویت اجرای عملگرها (Operator Precedence)

اولویت عملگرها مشخص می‌کند که در صورت وجود چندین عملگر در یک عبارت، کدام یک ابتدا اجرا می‌شود. اولویت‌ها در پایتون از بالا به پایین به شرح زیر هستند:

1. پرانتزها بالاترین اولویت را دارند.
2. عملگرهای ریاضی: به ترتیب (*, **, /, //, %, +, -)
3. عملگرهای مقایسه‌ای: <, <=, >, >=, !=, ==
4. عملگرهای منطقی: not, and, or
5. عملگرهای انتساب: =, +=, -=, *=, /=, //=, %=, **=

مقداردهی متغیرها و دستورنویسی پایتون

```
x = "Rahkar"
y = "is"
z = "your way"
print(x, y, z)
print(x + y + z)
x = 5
y = "Rahkar"
print(x)
print(y)
x = 1
y = 2.8
z = 1j
print(type(x))
print(type(y))
print(type(z))
```

آموزشگاه

```
Rahkar is your way
Rahkarisyour way
5
Rahkar
<class 'int'>
<class 'float'>
<class 'complex'>
```

```
x = 35e3
y = 12E4
z = -87.7e100
print((x))
print((y))
print((z))
print('----')
print(type(x))
print(type(y))
print(type(z))
```

```
35000.0
120000.0
-8.77e+101
----
<class 'float'>
<class 'float'>
<class 'float'>
```

کامپیوتر

```
x = 3+5j
y = 5j
z = -5j
print(x)
print(y)
print(z)
print(type(x))
print(type(y))
print(type(z))
```

ایجاد متغیر از نوع اعداد مرکب

```
(3+5j)
5j
(-0-5j)
<class 'complex'>
<class 'complex'>
<class 'complex'>
```

عملکرد تابع int():

```
x = int(1)
y = int(2.8)
z = int("3")
print(x)
print(y)
print(z)
```

```
1
2
3
```

آموزشگاه

کامپیوتر

عملکرد تابع float():

```
x = float(1)
y = float(2.8)
z = float("3")
w = float("4.2")
print(x)
print(y)
print(z)
print(w)
```

```
1.0
2.8
3.0
4.2
```

راهکار

عملکرد تابع str():

```
x = str("s1")
y = str(2)
z = str(3.0)
print(x)
print(y)
print(z)
```

```
s1
2
3.0
```

آموزشگاه

کامپیوتر

راهکار

کار با رشته‌ها در پایتون

فرمت‌بندی رشته‌ها با استفاده از f-strings در پایتون

در پایتون، f-strings یا formatted string literals یکی از بهترین و ساده‌ترین روش‌ها برای فرمت‌بندی رشته‌ها (strings) است. این قابلیت از پایتون 3.6 به بعد معرفی شد و به شما این امکان را می‌دهد که به راحتی مقادیر متغیرها را در داخل رشته‌ها قرار دهید بدون اینکه نیاز به استفاده از عملگرهای پیچیده یا توابع اضافی داشته باشید.

نحوه استفاده از f-strings

برای استفاده از f-strings، کافی است که رشته را با حرف f قبل از علامت نقل قول (single یا double quotes) شروع کنید. سپس می‌توانید داخل رشته، از آکولادها {} برای قرار دادن متغیرها یا عبارات استفاده کنید.

فرمت کلی:

"متن ثابت {متغیر یا عبارت}"

مثال‌های ساده از f-strings

1. استفاده از متغیر در f-string

فرض کنید دو متغیر داریم. برای وارد کردن این متغیرها داخل یک رشته، می‌توانیم از f-string به صورت زیر استفاده کنیم:

```
name = "Ali"
age = 25
text = f"شما {age} ساله هستید، {name} سلام"
print(text)
```

پاسخ:

شما 25 ساله هستید، Ali سلام

فرمت‌بندی اعداد در f-strings

یکی از ویژگی‌های جالب f-strings این است که می‌توانید برای فرمت‌بندی اعداد، تاریخ‌ها و مقادیر دیگر از قالب‌های مختلف استفاده کنید.

مثال 1: فرمت‌بندی اعداد اعشاری

```
pi = 3.14159265358979
formatted = f"عدد پی برابر است با {pi:.2f}"
print(formatted)
```

مثال 2: استفاده از فرمت برای اعداد بزرگ

```
large_number = 1000000
formatted = f"عدد به‌صورت فرمت هزارگان: {large_number:,}"
print(formatted)
```

مثال 3: فرمت تاریخ‌ها

```
from datetime import datetime
current_date = datetime.now()
formatted = f"تاریخ امروز: {current_date:%Y-%m-%d %H:%M:%S}"
print(formatted)
```

مزایای f-strings نسبت به روش‌های دیگر فرمت‌بندی

- خوانایی بالا f-strings به شما اجازه می‌دهند که مقادیر متغیرها را به سادگی داخل رشته‌ها قرار دهید.
- عملکرد سریع‌تر f-strings از عملکرد بهتری نسبت به روش‌های قدیمی‌تر مانند استفاده از str.format() دارند.
- انعطاف‌پذیری بالا: شما می‌توانید از عبارات، توابع و فرمت‌های پیچیده داخل f-strings استفاده کنید.

متدهای نوع داده رشته‌ای (String)

متد	توضیح	مثال
upper()	تمام حروف رشته را به حروف بزرگ تبدیل می‌کند.	<pre>text = "hello" print(text.upper()) # خروجی: 'HELLO'</pre>
lower()	تمام حروف رشته را به حروف کوچک تبدیل می‌کند.	<pre>text = "HELLO" print(text.lower()) # خروجی: 'hello'</pre>
capitalize()	اولین حرف رشته را بزرگ کرده و بقیه را کوچک می‌کند.	<pre>text = "hello world" print(text.capitalize()) # خروجی: 'Hello world'</pre>
title()	اولین حرف هر کلمه را بزرگ می‌کند.	<pre>text = "hello world" print(text.title()) # خروجی: 'Hello World'</pre>
strip()	فاصله‌های اضافی در ابتدا و انتهای رشته را حذف می‌کند.	<pre>text = " hello world " print(text.strip()) # خروجی: 'hello world'</pre>
lstrip()	فاصله‌های اضافی از ابتدای رشته را حذف می‌کند.	<pre>text = " hello world" print(text.lstrip()) # خروجی: 'hello world'</pre>
rstrip()	فاصله‌های اضافی از انتهای رشته را حذف می‌کند.	<pre>text = "hello world " print(text.rstrip()) # خروجی: 'hello world'</pre>
find()	موقعیت اولین وقوع یک زیررشته را می‌دهد. اگر پیدا نشد 1- باز می‌گرداند.	<pre>text = "hello world" print(text.find("world")) # خروجی: 6</pre>
replace()	یک زیررشته را با زیررشته‌ای دیگر جایگزین می‌کند.	<pre>text = "hello world" print(text.replace("world", "Python")) # خروجی: 'hello Python'</pre>
split()	رشته را بر اساس یک جداکننده مشخص تقسیم می‌کند و یک لیست برمی‌گرداند.	<pre>text = "apple,orange,banana" print(text.split(",")) # خروجی: ['apple', 'orange', 'banana']</pre>
join()	یک لیست از رشته‌ها را با جداکننده‌ای خاص به هم می‌چسباند.	<pre>words = ['hello', 'world'] print(" ".join(words)) # خروجی: 'hello world'</pre>
isalpha()	بررسی می‌کند که آیا رشته تنها شامل حروف است.	<pre>text = "hello" print(text.isalpha()) # خروجی: True</pre>
isdigit()	بررسی می‌کند که آیا رشته تنها شامل اعداد است.	<pre>text = "12345" print(text.isdigit()) # خروجی: True</pre>
isspace()	بررسی می‌کند که آیا رشته تنها شامل فضای خالی است.	<pre>text = " " print(text.isspace()) # خروجی: True</pre>
startswith()	بررسی می‌کند که آیا رشته با یک زیررشته خاص شروع می‌شود.	<pre>text = "hello world" print(text.startswith("hello")) # خروجی: True</pre>

مثال	توضیح	متد
<pre>text = "hello world" print(text.endswith("world"))</pre> # خروجی: True	بررسی می‌کند که آیا رشته با یک زیررشته خاص تمام می‌شود.	endswith()
<pre>text = "hello hello world" print(text.count("hello"))</pre> # خروجی: 2	تعداد وقوع یک زیررشته در رشته را می‌شمارد.	count()
<pre>text = "42" print(text.zfill(5))</pre> # خروجی: '00042'	صفرهای اضافی به ابتدای رشته اضافه می‌کند تا طول رشته به اندازه مشخصی برسد.	zfill()
<pre>text = "hello\tworld" print(text.expandtabs(4))</pre> # خروجی: 'hello world'	تمام کاراکترهای تب (\t) را به فضای سفید تبدیل می‌کند.	expandtabs()
<pre>text = "xxxhelloxxx" print(text.strip("x"))</pre> # خروجی: 'hello'	فضای خالی (whitespace) را از ابتدای و انتهای رشته حذف می‌کند.	strip()
<pre>text = "HELLO" print(text.isupper())</pre> # خروجی: True	بررسی می‌کند که آیا تمام حروف رشته به حروف بزرگ است.	isupper()
<pre>text = "hello" print(text.islower())</pre> # خروجی: True	بررسی می‌کند که آیا تمام حروف رشته به حروف کوچک است.	islower()
<pre>text = "hello123" print(text.isalnum())</pre> # خروجی: True	بررسی می‌کند که آیا رشته فقط شامل حروف و اعداد است.	isalnum()
<pre>text = "12345" print(text.isnumeric())</pre> # خروجی: True	بررسی می‌کند که آیا رشته تنها شامل اعداد است.	isnumeric()
<pre>text = "Hello World" print(text.istitle())</pre> # خروجی: True	بررسی می‌کند که آیا رشته مانند یک عنوان (با حروف بزرگ شروع شده و سایر حروف کوچک) است.	istitle()
<pre>text = "my_var" print(text.isidentifier())</pre> # خروجی: True	بررسی می‌کند که آیا رشته یک شناسه معتبر پایتون است (مثل نام متغیر).	isidentifier()
<pre>text = "Hello {name}, you are {age} years old" print(text.format(name="Ali", age=25))</pre> # خروجی: 'Hello Ali, you are 25 years old'	فرمت‌بندی رشته‌ها با استفاده از جایگذاری مقادیر.	format()

مبحث Slicing رشته‌ها در پایتون

Slicing یکی از قابلیت‌های قدرتمند در پایتون است که به شما این امکان را می‌دهد که بخشی از یک رشته (String) یا لیست (List) را استخراج کنید. این ویژگی در پایتون بسیار پرکاربرد است و به شما کمک می‌کند تا به راحتی از داده‌ها برش بزنید و قطعات دلخواه را بدست آورید.

نحوه استفاده از Slicing در پایتون

برای انجام Slicing در پایتون، باید از ترکیب ایندکس‌ها استفاده کنید. این فرمت کلی برای Slicing است:

```
string[start:end:step]
```

Start : ایندکس شروع. از کدام موقعیت در رشته (یا لیست) شروع کنیم.
End : ایندکس پایان. در واقع ایندکسی که نمی‌خواهید در آن گنجانده شود (نقطه‌ای که باید متوقف شوید).
Step : فاصله یا گام بین دو عنصر. به شما این امکان را می‌دهد که فقط از هر n امین کاراکتر استفاده کنید.

آموزشگاه

جزئیات پارامترهای Slicing :

- Start (ایندکس شروع): این پارامتر ایندکس اولین کاراکتر را که می‌خواهید در نتیجه باشد، مشخص می‌کند. اگر start را نگذارید، به طور پیش‌فرض از اولین کاراکتر رشته شروع می‌شود (ایندکس 0).
- End (ایندکس پایان): این پارامتر ایندکس آخرین کاراکتری که می‌خواهید شامل نتیجه باشد را مشخص می‌کند. این ایندکس شامل نمی‌شود (به عبارت دیگر، از ایندکس end به بعد هیچ چیزی شامل نمی‌شود).
- Step (گام): این پارامتر به شما اجازه می‌دهد که هر n امین کاراکتر را انتخاب کنید. اگر آن را نگذارید، به طور پیش‌فرض $step = 1$ است (یعنی همه کاراکترها به ترتیب گرفته می‌شوند).

آموزشگاه

مثال‌های کاربردی Slicing در پایتون

مثال 1: استخراج یک بخش از رشته

فرض کنید رشته‌ی زیر را داریم:

```
text = "Hello, Rahkar!"
```

دریافت اولین 5 کاراکتر:

```
print(text[:5]) # خروجی: 'Hello'
```

اینجا start به طور پیش‌فرض 0 است و از ابتدای رشته تا ایندکس 5 (غیر از آن) می‌رویم.

دریافت همه کاراکترها از ایندکس 7 تا 12:

```
print(text[7:12]) # خروجی: 'Rahka'
```

چون end را ننوشتیم، پایتون به طور پیش فرض تا آخر رشته ادامه می دهد.

دریافت همه کاراکترها با گام 2 (یعنی از هر 2 امین کاراکتر):

```
print(text[::2]) # خروجی: 'Hlo hk!'
```

در اینجا، از هر 2 امین کاراکتر یک کاراکتر انتخاب می شود.

مثال 2: استفاده از پارامتر step

گام معکوس (معکوس کردن رشته):

```
print(text[::-1]) # خروجی: '!rakahR ,olleH'
```

با استفاده از $step = -1$ ، رشته به طور معکوس برگردانده می شود.

انتخاب کاراکترها با گام 3:

```
print(text[::3]) # خروجی: 'Hl ak!'
```

در اینجا، از هر 3 امین کاراکتر یک کاراکتر انتخاب می شود.

سایر نکات مهم در Slicing:

- استفاده از ایندکس منفی: در پایتون، می توانید از ایندکس منفی برای اشاره به کاراکترها از انتهای رشته استفاده کنید. ایندکس منفی به این صورت عمل می کند:
- ایندکس 1- آخرین کاراکتر رشته است.
- ایندکس 2- دومین کاراکتر از انتها است و به همین ترتیب.

```
text='Hello Rahkar!'
```

```
print(text[-1]) # خروجی: '!'
```

```
print(text[-5:]) # خروجی: 'Rahkar!'
```

- استفاده از None در Slicing: شما می‌توانید به‌طور پیش‌فرض start, end و step را با None مشخص کنید.

- start = None به معنای شروع از اولین کاراکتر است.
- end = None به معنای رفتن تا انتهای رشته است.
- step = None به معنای استفاده از گام پیش‌فرض (1) است.

```
print(text[None:None:None]) # همان رشته اصلی 'Hello, Rahkar!'
```

جمع‌بندی

- Slicing در پایتون یک روش قدرتمند و مفید برای استخراج بخش‌هایی از یک رشته یا لیست است. می‌توانید با استفاده از پارامترهای start, end و step بخش‌های خاصی از داده‌ها را انتخاب کنید.
- اگر start و end را نگذارید، به‌طور پیش‌فرض از ابتدا تا انتها و با گام 1 در نظر گرفته می‌شود.
- ایندکس منفی به شما این امکان را می‌دهد که از انتهای رشته به جلو حرکت کنید.
- برای تغییر گام، می‌توانید از پارامتر step استفاده کنید.

آموزشگاه

کامپیوتر

راهکار

آموزشگاه

کامپیوتر

راهکار

کار با لیست‌ها در پایتون

لیست‌ها (Lists) در پایتون یکی از مهم‌ترین و پرکاربردترین ساختارهای داده هستند که برای ذخیره مجموعه‌ای از آیتم‌ها یا عناصر به صورت ترتیب‌دار (Ordered) استفاده می‌شوند. لیست‌ها می‌توانند شامل انواع مختلف داده‌ها باشند و تغییرپذیر هستند (یعنی می‌توانند بعد از ساخت، تغییر کنند).

ویژگی‌های لیست‌ها:

- ترتیب‌دار (Ordered): ترتیب عناصر در لیست حفظ می‌شود.
- تغییرپذیر (Mutable): می‌توانیم عناصر یک لیست را تغییر دهیم.
- اجزای مختلف: لیست‌ها می‌توانند شامل هر نوع داده‌ای (اعداد، رشته‌ها، لیست‌های دیگر، اشیاء و ...) باشند.
- دسترسی به عناصر: با استفاده از ایندکس می‌توانیم به عناصر لیست دسترسی پیدا کنیم.
- ایندکس منفی: می‌توان از ایندکس‌های منفی برای دسترسی به عناصر از انتهای لیست استفاده کرد.

تعریف لیست خالی

```
a=list()
```

```
b=[]
```

```
a = ["apple", "banana", "cherry"]
```

```
print(a)
```

```
['apple', 'banana', 'cherry']
```

دسترسی به آیتم‌های لیست

```
a = ["apple", "banana", "cherry"]
```

```
print(a[1])
```

```
banana
```

ایجاد لیست تو در تو

```
a = ["apple", "banana", ["cherry", "blackcurrant"], "orange"]
```

```
print(a)
```

```
print(a[2])
```

```
print(a[2][1])
```

```
['apple', 'banana', ['cherry', 'blackcurrant'], 'orange']
['cherry', 'blackcurrant']
blackcurrant
```

تغییر مقدار یکی از آیتم‌های لیست

```
a = ["apple", "banana", "cherry"]
```

```
a[1] = "blackcurrant"
```

```
print(a)
```

```
['apple', 'blackcurrant', 'cherry']
```

```
mylist = ["apple", "banana", "cherry"]
```

```
for x in mylist:
```

```
    print(x)
```

```
apple
banana
cherry
```

چاپ اعضای لیست با حلقه for

کنترل موجود بود یک آیتم داخل آیتم‌های لیست

mylist = ["apple", "banana", "cherry"]

if "apple" in mylist:

Yes, 'apple' is in the fruits list

print("Yes, 'apple' is in the fruits list")

استخراج طول لیست

mylist = ["apple", "banana", "cherry"]

print(len(mylist))

3

کامپیوتر

متدهای نوع داده List

راهکار

متد	توضیح	مثال
append()	افزودن یک عنصر به انتهای لیست	my_list = [1, 2, 3] my_list.append(4); print(my_list) # خروجی: [1, 2, 3, 4]
insert()	افزودن یک عنصر در موقعیت مشخص شده (ایندکس) در لیست	my_list = [1, 2, 3] my_list.insert(1, 'a') print(my_list) # خروجی: [1, 'a', 2, 3]
remove()	حذف اولین وقوع یک عنصر از لیست	my_list = [1, 2, 3, 2] my_list.remove(2) print(my_list) # خروجی: [1, 3, 2]
pop()	حذف و برگرداندن عنصر از انتهای لیست (یا ایندکس خاص)	my_list = [1, 2, 3] x = my_list.pop() print(x) # خروجی: 3 my_list.pop(0) print(my_list) # خروجی: [2, 3]
clear()	حذف تمام عناصر لیست	my_list = [1, 2, 3] my_list.clear() print(my_list) # خروجی: []
index()	یافتن ایندکس اولین وقوع یک عنصر در لیست	my_list = [1, 2, 3] print(my_list.index(2)) # خروجی: 1
count()	شمارش تعداد وقوع یک عنصر در لیست	my_list = [1, 2, 2, 3] print(my_list.count(2)) # خروجی: 2

مثال	توضیح	متد
<pre>my_list = [3, 1, 2] my_list.sort(); print(my_list) # خروجی: [1, 2, 3]</pre>	مرتب‌سازی لیست (به‌طور پیش‌فرض به ترتیب صعودی).	sort()
<pre>my_list = [1, 2, 3] my_list.reverse() print(my_list) # خروجی: [3, 2, 1]</pre>	معکوس کردن ترتیب عناصر لیست.	reverse()
<pre>my_list = [1, 2, 3] new_list = my_list.copy() print(new_list) # خروجی: [1, 2, 3]</pre>	ساخت یک کپی از لیست.	copy()
<pre>list1 = [1, 2] list2 = [3, 4] list1.extend(list2) print(list1) # خروجی: [1, 2, 3, 4]</pre>	افزودن تمام عناصر یک لیست به لیست دیگر.	extend()
<pre>my_list = [1, 2, 3] print(my_list.pop(0)) # خروجی: 1 print(my_list) # خروجی: [2, 3]</pre>	حذف و برگرداندن عنصر از لیست (با ایندکس دلخواه).	pop()
<pre>my_list = [1, 3, 2] my_list.sort(reverse=True) print(my_list) # خروجی: [3, 2, 1]</pre>	مرتب‌سازی لیست به صورت نزولی (یا معکوس).	sort(reverse=True)
<pre>my_list = [1, 2, 3] my_list.reverse() print(my_list) # خروجی: [3, 2, 1]</pre>	معکوس کردن لیست.	reverse()

توضیحات تکمیلی:

1. **append()**: فقط یک عنصر به انتهای لیست اضافه می‌کند.
2. **insert()**: عنصر را در ایندکس دلخواه در لیست اضافه می‌کند.
3. **remove()**: اولین وقوع از عنصر داده شده را از لیست حذف می‌کند.
4. **pop()**: یک عنصر را از انتهای لیست (یا ایندکس خاص) حذف کرده و برمی‌گرداند.
5. **clear()**: تمام عناصر لیست را حذف می‌کند.
6. **index()**: ایندکس اولین وقوع عنصر مورد نظر در لیست را برمی‌گرداند.
7. **count()**: تعداد دفعاتی که یک عنصر در لیست ظاهر شده است را برمی‌گرداند.
8. **sort()**: لیست را به‌طور پیش‌فرض به ترتیب صعودی مرتب می‌کند.
9. **reverse()**: ترتیب عناصر در لیست را معکوس می‌کند.
10. **copy()**: یک کپی جدید از لیست ایجاد می‌کند.
11. **extend()**: تمام عناصر یک لیست را به انتهای لیست دیگر اضافه می‌کند.

کار با تاپل‌ها در پایتون

تاپل (Tuple) در پایتون یکی از ساختارهای داده‌ای است که برای ذخیره مجموعه‌ای از داده‌ها استفاده می‌شود. مشابه لیست‌ها (lists)، اما با ویژگی‌های خاص خود که آن‌ها را از لیست‌ها متمایز می‌کند.

ویژگی‌های تاپل:

• غیر قابل تغییر: (Immutable)

برخلاف لیست‌ها، تاپل‌ها تغییرناپذیر هستند. به این معنی که پس از تعریف یک تاپل، نمی‌توانیم عناصر آن را تغییر دهیم، اضافه کنیم یا حذف کنیم.

• ترتیب‌دار: (Ordered)

تاپل‌ها مانند لیست‌ها ترتیب دارند. این به این معناست که می‌توانیم به هر عنصر دسترسی پیدا کرده و از ایندکس‌ها برای مراجعه به آن‌ها استفاده کنیم.

• دسترسی به عناصر از طریق ایندکس:

مانند لیست‌ها، می‌توانیم از ایندکس برای دسترسی به عناصر تاپل استفاده کنیم.

• محتویات مختلف:

تاپل می‌تواند از انواع داده‌های مختلف تشکیل شود (اعداد، رشته‌ها، لیست‌ها، و حتی دیگر تاپل‌ها).

• پشتیبانی از ایندکس منفی:

همانند لیست‌ها، می‌توان از ایندکس‌های منفی برای دسترسی به عناصر از انتهای تاپل استفاده کرد.

• پشتیبانی از عملیات‌هایی مثل طول‌سنجی، تکرار، و اتصال:

می‌توانید از عملیات‌هایی مانند `len()`, `+`, `*` برای انجام کارهای مختلف روی تاپل‌ها استفاده کنید.

• تاپل‌ها با استفاده از `()` تعریف می‌شوند، برخلاف لیست‌ها که با `[]` تعریف می‌شوند.

ایجاد تاپل خالی

```
t1 = ()
```

```
t2 = tuple()
```

```
t3 = (5,) # تعریف تاپل با یک عنصر
```

آموزشگاه

کامپیوتر

```
t1 = ("apple", "banana", "cherry")
```

```
print(t1)
```

```
('apple', 'banana', 'cherry')
```

دسترسی به آیتم‌های تاپل

```
t1 = ("apple", "banana", "cherry")
```

```
print(t1[1])
```

```
banana
```

تغییر مقدار یکی از ایندکس‌های تاپل

```
t1 = ("apple", "banana", "cherry")
```

```
t1[1] = "blackcurrant"
```

```
# مقادیر در تاپل، قابل تغییر نیستند
```

```
print(t1)
```

نمایش خطا بدلیل عدم امکان تغییر آیتم‌های تاپل

```
Traceback (most recent call last):
```

```
File "./prog.py", line 2, in <module>
```

```
TypeError: 'tuple' object does not support item assign
```

```
t1 = ("apple", "banana", "cherry")
for x in t1:
    print(x)
```

```
apple
banana
cherry
```

دسترسی به آیتم‌های تاپل با حلقه for

```
t1 = ("apple", "banana", "cherry")
if "apple" in t1:
    print("Yes, 'apple' is in the fruits tuple")
```

Yes, 'apple' is in the fruits tuple

کنترل موجود بودن یک مقدار داخل مقادیر تاپل

```
t1 = ("apple", "banana", "cherry")
print(len(t1))
```

3

محاسبه طول تاپل

```
thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple)
```

حذف کردن تاپل

این دستور موجب خطا میشه. چون تاپل دیگه وجود نداره

```
Traceback (most recent call last):
  File "demo_tuple_del.py", line 3, in <module>
    print(thistuple) #this will raise an error because the tuple no longer exists
NameError: name 'thistuple' is not defined
```

اتصال (Concatenation) و تکرار (Repetition)

• می‌توانید دو تاپل را با استفاده از عملگر + ترکیب کنید.

• می‌توانید یک تاپل را چند بار تکرار کنید با استفاده از علامت *

```
tuple1 = (1, 2)
tuple2 = (3, 4)
combined = tuple1 + tuple2
print(combined) # خروجی: (4, 3, 2, 1)

repeated = tuple1 * 3
print(repeated) # خروجی: (2, 1, 2, 1, 2, 1)
```

آموزشگاه

کامپیوتر

راهکار

تعداد و وجود (Count & Index)

• count(x) تعداد دفعاتی که x در تاپل آمده است را برمی‌گرداند.

• index(x) ایندکس اولین وقوع x را در تاپل می‌دهد.

```
my_tuple = (1, 2, 3, 2, 4)
print(my_tuple.count(2)) # خروجی: 2
print(my_tuple.index(3)) # خروجی: 2
```

مزایای استفاده از تاپل‌ها:

- عملکرد سریع‌تر نسبت به لیست‌ها: به دلیل تغییرناپذیری، تاپل‌ها معمولاً سریع‌تر از لیست‌ها هستند.
- اطمینان از عدم تغییر: در برخی مواقع، زمانی که نیاز دارید داده‌ها تغییر نکنند، از تاپل استفاده می‌شود.
- قابلیت استفاده به عنوان کلید دیکشنری: از آنجا که تاپل‌ها غیر قابل تغییر هستند، می‌توانند به عنوان کلید در دیکشنری‌ها استفاده شوند، در حالی که لیست‌ها نمی‌توانند چنین ویژگی‌ای داشته باشند.

کاربردهای رایج تاپل‌ها:

- نگهداری داده‌های ثابت: زمانی که می‌خواهید داده‌ها تغییر نکنند، از تاپل استفاده می‌کنید.
- بازگشت چندین مقدار از یک تابع: می‌توانید چندین مقدار را در قالب یک تاپل از یک تابع بازگردانید.
- استفاده به عنوان کلید دیکشنری: از تاپل‌ها می‌توان به عنوان کلید در دیکشنری‌ها استفاده کرد، در حالی که لیست‌ها چنین قابلیتی ندارند.

کار با set (مجموعه‌ها) در پایتون

در پایتون، مجموعه‌ها (Sets) یک نوع داده‌ای هستند که شامل مجموعه‌ای از اعضا (المان‌ها) می‌باشند و ویژگی‌های خاص خود را دارند. در اینجا به توضیح کامل مجموعه‌ها می‌پردازیم:

- یونیک بودن اعضا: در مجموعه‌ها هیچ عضو تکراری وجود ندارد. اگر یک عضو تکراری اضافه شود، تنها یک بار در مجموعه باقی می‌ماند.
- ترتیب ندارند: اعضای مجموعه‌ها ترتیب مشخصی ندارند و نمی‌توان از ایندکس‌گذاری برای دسترسی به عناصر استفاده کرد.
- قابلیت تغییر: مجموعه‌ها قابل تغییر هستند؛ یعنی می‌توان اعضای آن‌ها را اضافه یا حذف کرد.
- غیرقابل تغییر (immutable): مجموعه‌ها خودشان قابل تغییر هستند، اما اعضای آن‌ها می‌توانند از نوع داده‌هایی باشند که غیرقابل تغییر (immutable) باشند. به عنوان مثال، نمی‌توانید یک مجموعه از لیست‌ها بسازید.

ایجاد مجموعه

```
s1 = set()
s2 = {4, 1, 32, 15}
s3 = {15, 0, 26, (11, 5, 96), 80}
print(s1)
print(s2)
print(s3)
```

```
set()
{32, 1, 4, 15}
{0, 15, 80, (11, 5, 96), 26}
```

توجه: لیست مجموعه نامرتب است، به این معنی: موارد به ترتیب تصادفی ظاهر می‌شوند.

دسترسی به اعضای مجموعه با حلقه for

```
s1 = {"apple", "banana", "cherry"}
for x in s1:
    print(x)
```

```
apple
cherry
banana
```

کنترل موجود بودن یک مقدار در مجموعه

```
s1 = {"apple", "banana", "cherry"}
print("banana" in s1)
```

True

افزودن مقدار به مجموعه

```
s1 = {"apple", "banana", "cherry"}
s1.add("orange")
print(s1)
```

{'apple', 'banana', 'orange', 'cherry'}

افزودن چند مقدار در مجموعه

```
s1 = {"apple", "banana", "cherry"}
s1.update(["orange", "mango", "grapes"])
print(s1)
```

{'grapes', 'orange', 'apple', 'banana', 'mango', 'cherry'}

استخراج طول مجموعه

```
s1 = {"apple", "banana", "cherry"}
print(len(s1))
```

3

حذف مقدار از مجموعه

```
s1 = {"apple", "banana", "cherry"}
s1.remove("banana")
print(s1)
```

{'cherry', 'apple'}

حذف مقدار با متد discard()

```
s1 = {"apple", "banana", "cherry"}
s1.discard("banana")
print(s1)
```

{'apple', 'cherry'}

حذف آخرین آیتم با متد `pop()`

```
s1 = {"apple", "banana", "cherry"}
x = s1.pop()
print(x) # حذف آیتم
print(s1) # مجموعه بعد از حذف آیتم فوق
```

```
banana
{'apple', 'cherry'}
```

خالی کردن محتوای مجموعه

```
s1 = {"apple", "banana", "cherry"}
s1.clear()
print(s1)
```

`set()`

حذف کردن کل مجموعه

```
s1 = {"apple", "banana", "cherry"}
del s1
print(s1)
```

```
Traceback (most recent call last):
  File "demo_set_del.py", line 5, in <module>
    print(thisset) #this will raise an error because the set no longer exists
NameError: name 'thisset' is not defined
```

نمایش خطا بدلیل حذف شدن مجموعه `s1`اتحاد (Union) برای ترکیب دو مجموعه از عملگر `|` یا متد `union()` استفاده می شود.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1 | set2
print(result) # خروجی: {5, 4, 3, 2, 1}
```

اشتراک (Intersection): برای پیدا کردن اعضای مشترک دو مجموعه از عملگر `&` یا متد `intersection()`

استفاده می شود.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1 & set2
print(result) # خروجی: {3}
```

تفاضل (Difference): برای پیدا کردن اعضای یک مجموعه که در مجموعه دیگر نیستند از عملگر `-` یا متد`difference()` استفاده می شود.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1 - set2
print(result) # خروجی: {2, 1}
```

کاربردهای مجموعه‌ها:

- حذف عناصر تکراری: مجموعه‌ها به طور خودکار عناصر تکراری را حذف می‌کنند.
- عضویت سریع: عملیات بررسی عضویت (مثل `in` در مجموعه‌ها سریع‌تر از لیست‌ها هستند).
- عملیات ریاضی: مجموعه‌ها برای انجام عملیات ریاضی مانند اتحاد، اشتراک، و تفاضل بسیار مفید هستند.

آموزشگاه

کار با Dictionary در پایتون

کامپیوتر

در پایتون، دیکشنری (Dictionary) یک نوع داده‌ای است که برای ذخیره داده‌ها به صورت زوج کلید و مقدار (key-value) استفاده می‌شود. این داده‌ها به‌طور غیرترتیبی ذخیره می‌شوند، به این معنی که ترتیب خاصی برای ذخیره‌سازی آن‌ها وجود ندارد، اما هر کلید به‌طور منحصر به فرد به یک مقدار خاص اشاره می‌کند.

ویژگی‌های دیکشنری:

- زوج کلید-مقدار: هر دیکشنری از یک سری کلیدها و مقادیر تشکیل می‌شود. هر کلید باید یکتا باشد.
- قابلیت تغییر: دیکشنری‌ها قابل تغییر هستند؛ می‌توانید مقادیر را تغییر دهید یا کلیدهای جدید اضافه کنید.
- قابلیت استفاده از انواع داده‌های غیرقابل تغییر (immutable): برای کلیدها از انواع داده‌هایی که غیرقابل تغییرند (مثل رشته‌ها، اعداد، و تاپل‌ها) استفاده می‌شود.

نحوه ایجاد دیکشنری:

برای ایجاد یک دیکشنری، می‌توان از `{ }` استفاده کرد و کلیدها و مقادیر را با استفاده از `:` از هم جدا کرد.

راهکار

تعریف دیکشنری جدید

```
dict1 = { }
dict2 = dict()
print(dict1)
print(dict2)
```

```
{ }
{ }
```

آموزشگاه

تعریف Dictionary

```
# دیکشنری با استفاده از {}
my_dict = {'name': 'Ali', 'age': 25, 'city': 'Tehran'}

# دیکشنری خالی
empty_dict = {}

# ایجاد دیکشنری با استفاده از تابع dict()
my_dict2 = dict(name='Sara', age=30)
```

کامپیوتر
راهکار

دسترسی به آیتم‌های دیکشنری:

دسترسی به مقدار با استفاده از کلید: برای دسترسی به مقدار یک کلید از دیکشنری می‌توانید از سینتکس dict[key] استفاده کنید.

```
print(my_dict['name']) # خروجی: Ali
```

آموزشگاه

اضافه کردن یا تغییر مقدار: اگر کلید جدیدی اضافه شود، یک زوج کلید-مقدار جدید در دیکشنری افزوده می‌شود. همچنین اگر کلید قبلاً وجود داشته باشد، مقدار آن تغییر می‌کند.

```
my_dict['age'] = 26 # تغییر مقدار
my_dict['country'] = 'Iran' # اضافه کردن یک کلید جدید
```

حذف مقدار: برای حذف یک کلید و مقدار آن از دیکشنری از del یا متد pop() می‌توان استفاده کرد.

del: حذف یک کلید از دیکشنری

آموزشگاه

pop(): حذف یک کلید و بازگشت مقدار آن

```
del my_dict['city'] # حذف کلید 'city'
removed_value = my_dict.pop('age') # دریافت مقدار آن 'age' حذف کلید
print(removed_value) # خروجی: 26
```

راهکار

دریافت کلیدها، مقادیر و زوجها: برای دریافت تمامی کلیدها، مقادیر یا زوج‌های کلید-مقدار می‌توان از متدهای keys(), values(), items() استفاده کرد.

```
print(my_dict.keys()) # خروجی: dict_keys(['name', 'country'])
print(my_dict.values()) # خروجی: dict_values(['Ali', 'Iran'])
print(my_dict.items()) # خروجی: dict_items([('name', 'Ali'), ('country', 'Iran')])
```

کامپیوتر

بررسی وجود کلید: برای بررسی اینکه آیا یک کلید در دیکشنری وجود دارد یا نه، از کلمه کلیدی in استفاده می‌شود.

راهکار

```
print('name' in my_dict) # خروجی: True
print('city' in my_dict) # خروجی: False
```

پاک کردن تمام عناصر: برای پاک کردن تمامی کلیدها و مقادیر از دیکشنری از متد clear() استفاده می‌شود.

```
my_dict.clear()
print(my_dict) # خروجی: {}
```

کاربردهای دیکشنری در پایتون:

- ذخیره‌سازی داده‌های مرتبط: دیکشنری‌ها به‌طور خاص برای ذخیره‌سازی داده‌هایی که به یکدیگر مرتبط هستند (مثل نام و سن یک شخص) مناسب‌اند.
- یافتن سریع داده‌ها: دیکشنری‌ها برای جستجوی سریع داده‌ها بسیار مناسب هستند زیرا به‌طور داخلی از روش‌های جستجو مانند هش استفاده می‌کنند.
- ایجاد شمارنده‌ها و آمار: دیکشنری‌ها برای شمارش تعداد تکرارهای یک مقدار خاص (مانند تعداد دفعات وقوع یک کلمه در یک متن) مفید هستند.
- ایجاد ساختارهای پیچیده: دیکشنری‌ها می‌توانند به عنوان اجزای دیگر دیکشنری‌ها یا لیست‌ها برای ایجاد ساختارهای داده‌ای پیچیده‌تر استفاده شوند.

بررسی شرط‌ها در پایتون

در پایتون، شرط‌ها (Conditions) به شما این امکان را می‌دهند که بر اساس شرایط مختلف، تصمیم بگیرید که کدام کد اجرا شود. به عبارت دیگر، شرط‌ها به شما این اجازه را می‌دهند که در برنامه‌نویسی مسیرهای مختلفی را طبق تصمیماتی که برنامه می‌گیرد، طی کنید. مهم‌ترین ابزارها برای پیاده‌سازی شرط‌ها در پایتون، عبارت‌های **if** و **else** و **elif** هستند.

استفاده از عبارت **if**: عبارت **if** برای بررسی یک شرط و اجرای بلوک کد خاص در صورتی که آن شرط درست باشد استفاده می‌شود.

شرط **if**:

کدهایی که در صورت درست بودن شرط اجرا می‌شوند #

```
x = 10
```

```
if x > 5:
```

```
    print("بزرگتر از 5 است x")
```

در این مثال، چون شرط $x > 5$ درست است، پیامی مبنی بر بزرگتر بودن x از 5 چاپ خواهد شد.

استفاده از عبارت **else**

عبارت **else** به معنای "در غیر این صورت" است. این بخش از کد زمانی اجرا می‌شود که شرط **if** غلط باشد.

```

شرط:
    کدهایی که در صورت درست بودن شرط اجرا می‌شوند
else:
    کدهایی که در صورت غلط بودن شرط اجرا می‌شوند

x = 3
if x > 5:
    print("بزرگتر از 5 است x")
else:
    print("بزرگتر از 5 نیست x")
  
```

آموزشگاه

کامپیوتر

راهکار

در این مثال، چون شرط $x > 5$ غلط است، پیامی مبنی بر نادرست بودن آن چاپ خواهد شد.

آموزشگاه

استفاده از عبارت **elif (else if)**

اگر نیاز داشته باشید که بیش از دو حالت را بررسی کنید (یعنی چند شرط مختلف را به ترتیب بررسی کنید)، می‌توانید از عبارت **elif** استفاده کنید **elif**. به شما این امکان را می‌دهد که شرایط دیگری را بعد از **if** بررسی کنید.

```

شرط 1:
    کدهایی که در صورت درست بودن شرط 1 اجرا می‌شوند
elif شرط 2:
    کدهایی که در صورت درست بودن شرط 2 اجرا می‌شوند
else:
    کدهایی که در صورت غلط بودن تمام شرایط اجرا می‌شوند
  
```

آموزشگاه

کامپیوتر

راهکار

```

x = 7
if x > 10:
    print("بزرگتر از 10 است x")
elif x == 7:
    print("برابر با 7 است x")
else:
    print("کوچکتر از 7 است x")
  
```

در این مثال، چون شرط اول غلط است و شرط **elif** درست است، پیامی مبنی بر برابر بودن x با ۷ چاپ می‌شود.

شرط‌های پیچیده:

شما می‌توانید از عملگرهای منطقی مانند `and`, `or`, و `not` برای ترکیب چند شرط استفاده کنید.

- **استفاده از `and`:** عملگر `and` زمانی استفاده می‌شود که می‌خواهید چند شرط را همزمان بررسی کنید و همه آن‌ها باید درست باشند تا کد اجرا شود.

```
x = 10
y = 5
if x > 5 and y < 10:
    print("هر دو شرط درست هستند")
```

آموزشگاه

کامپیوتر

راهکار

در این مثال، چون هر دو شرط ($x > 5$ و $y < 10$) درست هستند، پیامی مبنی بر درست بودن هر دو شرط چاپ خواهد شد.

- **استفاده از `or`:** عملگر `or` زمانی استفاده می‌شود که فقط یکی از چند شرط باید درست باشد تا کد اجرا شود.

```
x = 3
y = 8
if x > 5 or y < 10:
    print("حداقل یکی از شرایط درست است")
```

راهکار

در این مثال، چون شرط $y < 10$ درست است، پیامی مبنی بر درست بودن حداقل یکی از شرایط چاپ می‌شود.

- **استفاده از `not`:** عملگر `not` برای معکوس کردن نتیجه یک شرط استفاده می‌شود. اگر شرط درست باشد، `not` آن را غلط می‌کند و بالعکس.

کامپیوتر

راهکار

```
x = 3
if not x > 5:
    print("غلط است x > 5 شرط")
```

در این مثال، چون $x > 5$ غلط است، `not` آن را درست می‌کند و پیامی مبنی بر غلط بودن شرط $x > 5$ چاپ می‌شود.

شرط‌های تو در تو (Nested if) :

شما می‌توانید از شرط‌ها در داخل شرط‌های دیگر استفاده کنید. این نوع شرط‌ها به نام شرط‌های تو در تو شناخته می‌شوند.

```
x = 7
if x > 5:
    if x == 7:
        print("برابر با 7 است x")
    else:
        print("بزرگتر از 5 است، اما برابر با 7 نیست x")
else:
    print("کوچکتر یا مساوی 5 است x")
```

آموزشگاه

کامپیوتر

راهکار

در این مثال، چون شرط اول ($x > 5$) درست است، وارد شرط داخلی می‌شود و چون $x == 7$ هم درست است، پیامی مبنی بر برابر بودن x با 7 چاپ می‌شود.

آموزشگاه

کامپیوتر

راهکار

جمع‌بندی:

- **if** : برای بررسی یک شرط و اجرای کد در صورت درست بودن آن.
- **else** : برای اجرای کد در صورتی که شرط **if** غلط باشد.
- **elif** : برای بررسی چندین شرط به ترتیب.
- عملگرهای منطقی: (**and, or, not**) برای ترکیب یا معکوس کردن چند شرط.
- شرط‌های تو در تو: برای بررسی شرایط پیچیده‌تر با استفاده از شرط‌های داخل هم.

کامپیوتر

راهکار

کار با حلقه‌های تکرار (Loop) در پایتون

```
i = 1
while i < 6:
    print(i)
    i += 1
```

```
1
2
3
4
5
```

کار با حلقه while

استفاده از دستور break در داخل حلقه while

```
i = 1
while i < 6:
    print(i)
    if (i == 3):
        break
    i += 1
```

```
1
2
3
```

استفاده از دستور continue در داخل حلقه while

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
1
2
4
5
6
```

عدد 3 در خروجی چاپ نشده است

کار با حلقه for برای دسترسی به آیتم‌های لیست

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

```
apple
banana
cherry
```

کار با حلقه for برای دسترسی به کاراکترهای رشته

```
for x in "banana":
    print(x)
```

```
b
a
n
a
n
a
```

استفاده از دستور break در حلقه for

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

```
apple
banana
```

استفاده از دستور continue در حلقه for

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

```
apple
cherry
```

استفاده از تابع range() در حلقه for

```
for x in range(6):
    print(x)
```

```
0
1
2
3
4
5
```

range(start)
range(start , end)
range(start , end , step)

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```

```
0  
1  
2  
3  
4  
5  
Finally finished!
```

استفاده از else در حلقه for

آموزشگاه

```
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]  
for x in adj:  
    for y in fruits:  
        print(x, y)
```

```
red apple  
red banana  
red cherry  
big apple  
big banana  
big cherry  
tasty apple  
tasty banana  
tasty cherry
```

تعریف حلقه for تو در تو

راهکار

آموزشگاه

کامپیوتر

راهکار

آموزشگاه

کامپیوتر

راهکار

توابع (Function) در پایتون

ایجاد و فراخوانی تابع

```
def my_function():
    print("Hello from a function")
```

```
Hello from a function
```

```
my_function()
```

استفاده از آرگومان در فراخوانی تابع

```
def my_function(fname):
    print(fname + " Refsnes")
```

```
Emil Refsnes
Tobias Refsnes
Linus Refsnes
```

```
my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

استفاده از آرگومان پیش فرض در فراخوانی تابع

```
def my_function(country = "Iran "):
    print("I am from " + country)
```

```
my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

```
I am from Sweden
I am from India
I am from Iran
I am from Brazil
```

برگرداندن یک مقدار بعنوان خروجی تابع

```
def my_function(x):
    return 5 * x
print(my_function(3))
print(my_function(5))
print(my_function(9))
```

```
15
25
45
```

تعریف تابع بازگشتی (recursion): فراخوانی مجدد تابع از درون خودش

```
def tri_recursion(k):
    if(k > 0):
        result = k + tri_recursion(k - 1)
        print(result)
    else:
        result = 0
    return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

```
Recursion Example Results
1
3
6
10
15
21
```

ایجاد تابع داخل تابع دیگر (Function inside Function)

در پایتون می‌توانیم یک تابع را داخل تابع دیگر تعریف کنیم. این ویژگی کاربردهای مختلفی دارد و می‌تواند به ساخت کدهای ماژولار و خوانا کمک کند.

تعریف تابع داخل تابع:

آموزشگاه

کامپیوتر

خروجی:

این تابع بیرونی است
این تابع درونی است

```
def outer_function():
    print("این تابع بیرونی است")

    def inner_function():
        print("این تابع درونی است")

    inner_function() # فراخوانی تابع درونی

outer_function() # فراخوانی تابع بیرونی
```

ویژگی‌های توابع درونی:

(1) محدوده دید (Scope):

تابع درونی فقط در داخل تابع بیرونی قابل دسترسی است.

```
def outer():
    def inner():
        print("Inner function")

    inner() # کار می‌کند

outer()
inner() # خطا: NameError: name 'inner' is not defined
```

آموزشگاه

(2) دسترسی به متغیرهای تابع

بیرونی: تابع درونی می‌تواند به متغیرهای تابع بیرونی دسترسی داشته باشد.

راهکار

```
def outer():
    message = "Hello"

    def inner():
        print(message) # دسترسی به متغیر تابع بیرونی

    inner()

outer() # خروجی: Hello
```

```
def outer(msg):
    def inner():
        print(msg)
    return inner

my_func = outer("سلام")
my_func() # خروجی: سلام
```

(3) بستار (Closure): توابع درونی می‌توانند حتی پس از پایان اجرای تابع بیرونی نیز به متغیرهای آن دسترسی داشته باشند.

آموزشگاه

کامپیوتر

کاربردهای توابع درونی

1. کپسوله‌سازی (Encapsulation)

2. ایجاد دکوراتورها (Decorators)

3. پیاده‌سازی الگوهای طراحی مانند Factory

4. کمک به خوانایی و سازماندهی کد

آموزشگاه

محدوده سراسری (Global Scope) در پایتون

محدوده سراسری در پایتون به متغیرها و توابعی اشاره دارد که در سطح مازول (فایل پایتون) تعریف شده‌اند و از هر جای کد قابل دسترسی هستند.

```
x = 10 # متغیر سراسری

def show_x():
    print(x) # دسترسی به متغیر سراسری

show_x() # خروجی: 10
print(x) # خروجی: 10
```

تعریف متغیر سراسری

آموزشگاه

کامپیوتر

تفاوت بین متغیرهای سراسری و محلی

```
y = 20 # متغیر سراسری

def test():
    y = 30 # متغیر محلی
    print("داخل تابع:", y) # 30

test()
print("خارج تابع:", y) # 20
```

```
count = 0

def increment():
    global count
    count += 1
    print("داخل تابع:", count)

increment() # 1: داخل تابع
increment() # 2: داخل تابع
print("خارج تابع:", count) # 2: خارج تابع
```

تغییر متغیر سراسری داخل تابع

برای تغییر مقدار یک متغیر
سراسری داخل یک تابع، باید از
کلمه کلیدی global استفاده

کنید آموزشگاه

کامپیوتر

راهکار

نکات مهم درباره محدوده سراسری

دسترسی: متغیرهای سراسری از هر جای ماژول (فایل پایتون) قابل دسترسی هستند.

تغییر: برای تغییر مقدار آنها در توابع باید از global استفاده کنید.

بهترین روش: استفاده زیاد از متغیرهای سراسری می تواند کد را پیچیده کند. بهتر است تا حد امکان از متغیرهای محلی استفاده شود.

نام گذاری: برای تشخیص بهتر، نام متغیرهای سراسری را معمولاً با حروف بزرگ می نویسند (مثل MAX_VALUE, CONFIG)

آموزشگاه

کامپیوتر

راهکار

تابع lambda در پایتون

تعریف تابع lambda

Lambda_name = lambda arguments: expression

افزودن ده واحد به پارامتر a از طریق تابع lambda

```
x = lambda a: a + 10
print(x(5))
```

15

استفاده از تابع lambda برای ضرب دو آرگومان در یکدیگر

```
x = lambda a, b: a * b
print(x(5, 6))
```

30

محاسبه توان دوم پارامتر از طریق تابع lambda

```
square = lambda x: x**2
print(square(5))
```

25

استفاده از تابع lambda برای مرتب سازی آیتم های لیست

```
names = ["Alice", "Bob", "Charlie", "David"]
sorted_names = sorted(names, key=lambda x: len(x))
print(sorted_names)
```

['Bob', 'Alice', 'David', 'Charlie']

مقایسه تعریف تابع با def یا lambda

```
def add(x, y):
    return x + y
# Call the function
print(add(2, 3))
-----
```

5

```
add = lambda x, y : x + y
print(add(2, 3))
```

5

تابع map در پایتون

یک تابع را بعنوان آرگومان اول دریافت و آن را بر روی تمام توالی‌های آرگومان دوم و ... اعمال می‌کند

```
map_name = map(function_object, iterable1, iterable2,...)
print(list( map_name ))
```

خروجی map به تنهایی قابل چاپ نیست و می‌توان آن را تبدیل به نوع لیست کرده و بعد چاپ کرد

آموزشگاه

محاسبه دوبرابر اعداد یک لیست توسط تابع map

```
def multiply2(x):
    return x * 2
```

```
a = map(multiply2, [1, 2, 3, 4, 2])
print(list(a))
```

[2, 4, 6, 8, 4]

راهکار

حل مثال قبل با ترکیب توابع map و lambda

```
a = map(lambda x : x*2, [1, 2, 3, 4, 2])
print(list(a))
```

[2, 4, 6, 8, 4]

آموزشگاه

پیمایش دیکشنری با استفاده از توابع map و lambda

```
dict_a = [{'name': 'python', 'points': 10}, {'name': 'java', 'points': 8}]
```

```
a = map(lambda x : x['name'], dict_a)
b = map(lambda x : x['points']*10, dict_a)
c = map(lambda x : x['name'] == "python", dict_a)
```

```
print(list(a))
print(list(b))
print(list(c))
```

```
['python', 'java']
[100, 80]
[True, False]
```

کامپیوتر
راهکار

پیمایش چندین توالی با استفاده از تابع map

```
list_a = [1, 2, 3]
list_b = [10, 20, 30]
```

```
a = map(lambda x, y: x + y, list_a, list_b)
print(list(a))
```

[11, 22, 33]

کامپیوتر

راهکار

تابع filter در پایتون

برای فیلتر کردن مقادیر مورد نظرمان از یک توالی این تابع بکار می‌رود

```
filter_name = filter( function_ object , iterable )
```

یک تابع را بعنوان آرگومان اول دریافت و آن را بر روی تمام توالی‌های آرگومان دوم اعمال می‌کند و خروجی باید برای هر کدام از مقادیر توالی از نوع Boolean (True / False) باشد

چاپ اعداد زوج از یک لیست

```
a = [1, 2, 3, 4, 5, 6]
f = filter(lambda x : x % 2 == 0, a)
print(list(f))
```

```
[2, 4, 6]
```

فیلتر کردن مقدار python از میان مقادیر دیکشنری

```
dict_a = [{'name': 'python', 'points': 10}, {'name': 'java', 'points': 8}]
```

```
f = filter(lambda x : x['name'] == 'python', dict_a)
print(list(f))
```

```
[{'name': 'python', 'points': 10}]
```

تابع zip() در پایتون

تابع zip() یکی از توابع داخلی پایتون است که برای ترکیب چندین دنباله (مانند لیست، تاپل، رشته و ...) استفاده می‌شود. این تابع یک آبجکت ایتريبل (قابل پیمایش) برمی‌گرداند که هر عنصر آن شامل ترکیب عناصر متناظر از دنباله‌های ورودی است. ساختار پایه :

```
zip(iterable1, iterable2, ...)
```

مثال ساده:

```
names = ['علی', 'رضا', 'سارا']
scores = [18, 20, 19]
```

```
zipped = zip(names, scores)
print(list(zipped))
```

```
[('علی', 18), ('رضا', 20), ('سارا', 19)]
```

کاربردهای عملی تابع zip() :

1. ایجاد دیکشنری از دو لیست
2. پیمایش همزمان چند لیست
3. مقایسه عناصر متناظر در دو لیست
4. چرخش ماتریس (جا به جایی سطر و ستون)

شی‌گرایی در پایتون (OOP)

تعریف کلاس

```
class MyClass:
    x = 5
```

```
<class '__main__.MyClass'>
```

```
print(MyClass)
```

استفاده مستقیم از نام کلاس مطابق دستور فوق امکان پذیر نیست

```
class MyClass:
    x = 5
```

```
p1 = MyClass()
print(p1.x)
```

ساخت شی از یک کلاس و دسترسی به خصوصیات آن

کامپیوتر

راهکار

استفاده از تابع سازنده در کلاس‌های پایتون و مقدار دهی اولیه خصوصیات

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
John
36
```

```
p1 = Person("John", 36)
print(p1.name)
print(p1.age)
```

کامپیوتر

ساخت متد داخل کلاس

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def myfunc(self):
        print("Hello my name is " + self.name)
```

راهکار

آموزشگاه

```
p1 = Person("John", 36)
p1.myfunc()
```

```
Hello my name is John
```

کامپیوتر

تغییر دادن اسم پارامتر self در داخل کلاس

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc):
        print("Hello my name is " + abc.name)
```

راهکار

```
p1 = Person("John", 36)
p1.myfunc()
```

```
Hello my name is John
```

دسترسی به ویژگی‌های کلاس و ویرایش آن

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)
```

```
p1 = Person("John", 36)
p1.age = 40
print(p1.age)
```

40

آموزشگاه

کامپیوتر

حذف ویژگی کلاس

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)
```

```
Traceback (most recent call last):
  File "demo_class7.py", line 13, in <module>
    print(p1.age)
AttributeError: 'Person' object has no attribute 'age'
```

```
p1 = Person("John", 36)
del p1.age
print(p1.age)
```

نمایش پیام خطا چون آن ویژگی پاک شده و قابل استفاده نیست

حذف کامل شی ایجاد شده از یک کلاس

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)
```

```
Traceback (most recent call last):
  File "demo_class8.py", line 13, in <module>
    print(p1)
NameError: 'p1' is not defined
```

```
p1 = Person("John", 36)
del p1.age
print(p1.age)
```

نمایش پیام خطا چون آن شی پاک شده و قابل استفاده نیست

کامپیوتر

راهکار

متد `__str__()` در داخل کلاس:

```
class Person:
    def __init__(self, name):
        self.name = name
```

```
p1 = Person('Rahkar')
print(p1)
```

<__main__.Person object at 0x155514e84130>

این متد برای زمانی به کار می‌رود که بخواهیم از شی تعریف شده چاپ بگیریم. ولی نتیجه چندان مطلوب نیست و آدرس حافظه مربوط به شی برایمان چاپ می‌شود. برای تنظیم مقدار دلخواه برای زمان چاپ شی از این متد مانند مثال زیر استفاده می‌کنیم.

```
class Person:
    def __init__(self, name):
        self.name = name
```

```
    def __str__(self):
        return self.name
```

```
p1 = Person('Rahkar')
print(p1)
```

Rahkar

متد `__repr__()` در داخل کلاس:

این متد هم مانند متد `__str__` می‌باشد با یک فرق کوچک که معمولاً متد `__repr__` برای زمانی است که بخواهیم خروجی را از طریق ترمینال نشان دهیم و معمولاً مخاطب ما یک برنامه نویس است ولی `__str__` معمولاً زمانی استفاده می‌شود که بخواهیم خروجی را به کاربر نمایش دهیم.

```
import datetime
now = datetime.datetime.now()
print(repr(now))
print(str(now))
```

**datetime.datetime(2024, 8, 19, 15, 31, 0, 410277)
2024-08-19 15:31:00.410277**

در کد بالا ما زمان فعلی رو ریختیم داخل متغیر `now` زمانی که با استفاده از متد `str` متغیر `now` رو صدا می‌زنیم می‌بینید که نتیجه به شکل یک زمان نشون داده میشه. اما زمانی که متغیر `now` رو با متد `repr` صدا می‌زنیم نتیجه رو به شکل یک فانکشن میاره. تفاوت این دو متد اینجا مشخص میشه. متد `str` برای نشون دادن یک نتیجه به کاربر عادی اما متد `repr` با هدف دیباگ کردن برنامه و نشون دادن یک پیغام به برنامه نویس دیگه استفاده میشه.

انواع متدها در کلاس‌های پایتون

سه نوع اصلی متدها در پایتون شامل:

1. متدهای نمونه (و هله‌ای یا instance method): اولین آرگومان متد self می‌باشد
2. متدهای کلاس (class method): اولین آرگومان متد cls می‌باشد
3. متدهای استاتیک (static method): برخلاف متدهای instance و class هیچ مقداری را بعنوان اولین آرگومان دریافت نمی‌کند

چهار اصل برنامه‌نویسی شی‌گرا

آموزشگاه

1. وراثت (inheritance)
2. چندریختی (polymorphism)
3. کپسوله‌سازی (encapsulation)
4. انتزاع (abstraction)

کامپیوتر

سطوح دسترسی به اعضای کلاس

1. نوع عمومی (Public): متغیر یا تابع کلاس در سایر کلاس‌ها یا نمونه‌های ایجاد شده از کلاس قابل دسترسی است. همه متدها و متغیرهای کلاس بصورت پیش‌فرض از نوع عمومی می‌باشند.
2. نوع محافظت شده (Protected): در این نوع دسترسی متغیر یا متد، در کلاسی که داخل آن تعریف شده یا کلاس‌های فرزند قابل دسترسی خواهد بود. برای تعریف ای نوع در ابتدای نام آن یک علامت _ قرار می‌دهیم
3. نوع خصوصی (Private): در این نوع از دسترسی متغیر یا تابع فقط در کلاسی که در آن تعریف شده قابل دسترسی است. هر متغیر یا متدی که با دو علامت __ شروع یا پایان یابد از نوع خصوصی می‌باشد.

آموزشگاه

کامپیوتر

راهکار

کار با ماژول‌ها در پایتون

افزودن ماژول کاربر به پروژه جاری و استفاده از آن

File1.py

```
def welcome(name):
    print('hi ', name)
```

File2.py

```
Import File1
Print(File1.welcome('David'))
```

hi David

استفاده از ماژول‌های داخلی پایتون

```
import platform
import random
```

```
x = platform.system()
print(x)
```

```
y=random.randint(10,20)
print(y)
```

Linux
15

تغییر دادن نام ماژول

```
import random as rnd
import File1 as f1
```

Import from Module در پایتون:

در پایتون، می‌توانید به جای وارد کردن کل یک ماژول، فقط بخش‌های خاصی از آن را import کنید. این روش مزایای متعددی دارد از جمله:

```
from math import sqrt, pow, sin
```

- کاهش مصرف حافظه (فقط آنچه نیاز دارید import می‌شود)
- خوانایی بهتر کد (مشخص است دقیقاً چه چیزی از ماژول استفاده می‌شود)
- جلوگیری از تداخل نام‌ها (namespace pollution)

تابع dir در پایتون کلیه ویژگی‌های یک آبجکت را نشان می‌دهد

```
x = 'David'  
print(dir(x))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__',  
 '__doc__', '__eq__', '__format__', '__ge__', '__getattr__',  
 '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__',  
 '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__',  
 '__mod__', '__mul__', '__ne__', '__new__', '__reduce__',  
 '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__',  
 '__sizeof__', '__str__', '__subclasshook__', 'capitalize',  
 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs',  
 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha',  
 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower',  
 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper',  
 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition',  
 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit',  
 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase',  
 'title', 'translate', 'upper', 'zfill']
```

آموزشگاه

کامپیوتر

راهکار

آموزشگاه

کامپیوتر

راهکار

کار با ماژول Date / Time

نمایش زمان جاری سیستم

```
import datetime
x = datetime.datetime.now()
print(x)
```

2024-08-14 12:15:33.355450

نمایش سال جاری و نام روز

```
import datetime
x = datetime.datetime.now()
print(x.year)
print(x.strftime("%A"))
```

2024
Wednesday

کار با ماژول های تاریخ و زمان در پایتون

پایتون ماژول های قدرتمندی برای کار با تاریخ و زمان ارائه می دهد. در اینجا به بررسی مهم ترین ماژول ها و کاربردهای آنها می پردازیم.

1. ماژول datetime: اصلی ترین ماژول برای کار با تاریخ و زمان در پایتون است.

کلاس های اصلی:

- Date: فقط تاریخ (سال، ماه، روز)
- Time: فقط زمان (ساعت، دقیقه، ثانیه، میکروثانیه)
- Datetime: ترکیب تاریخ و زمان
- Timedelta: تفاضل بین دو تاریخ/زمان
- Tzinfo: پایه برای کار با مناطق زمانی

ایجاد تاریخ و زمان:

```
from datetime import date, time, datetime

# فقط تاریخ
today = date.today()
print(today) # 2023-11-15

# فقط زمان
current_time = time(12, 30, 45)
print(current_time) # 12:30:45

# تاریخ و زمان
now = datetime.now()
print(now) # 2023-11-15 12:30:45.123456
```

PIP در پایتون: مدیریت بسته‌های پایتون

PIP مخفف (Pip Installs Packages) یک سیستم مدیریت بسته‌های نرم‌افزاری برای پایتون است که به شما امکان نصب، به‌روزرسانی و حذف کتابخانه‌ها و ماژول‌های پایتون را می‌دهد.

نکات کلیدی درباره PIP

- نصب خودکار: از نسخه پایتون 3.4 به بعد، PIP به صورت پیش‌فرض نصب می‌شود.
- مخزن بسته‌ها: PIP بسته‌ها را از PyPI (مخزن بسته‌های پایتون) دانلود می‌کند.
- وابستگی‌ها: به صورت خودکار وابستگی‌های هر بسته را نصب می‌کند.

نصب یک بسته:

نام بسته pip install

```
pip install requests
pip install numpy pandas matplotlib # نصب چند بسته با هم
```

```
pip install django==3.2.8
```

نصب نسخه خاصی از یک بسته:

```
pip install --upgrade requests
```

به‌روزرسانی یک بسته:

```
pip uninstall flask
```

حذف یک بسته:

ایجاد فایل اجرایی (EXE) از کد پایتون

برای تبدیل اسکریپت‌های پایتون به فایل اجرایی (EXE) که بدون نیاز به نصب پایتون قابل اجرا باشند، می‌توانید از ابزارهای زیر استفاده کنید:

```
pip install pyinstaller
```

1. استفاده از PyInstaller:

```
pyinstaller --onefile --windowed your_script.py
```

- `--onefile`: ایجاد یک فایل اجرایی واحد
- `--windowed`: جلوگیری از نمایش کنسول (برای برنامه‌های گرافیکی)
- `--icon=your_icon.ico`: اضافه کردن آیکون به فایل اجرایی

```
pyinstaller --onefile --windowed --icon=app.ico main.py
```

مثال:

مدیریت خطاها در پایتون با try-except

در پایتون، ساختار try-except برای مدیریت خطاها (استثناها) استفاده می‌شود. این ساختار به شما امکان می‌دهد تا خطاهای احتمالی را پیش‌بینی و کنترل کنید.

```
try:
    # کدی که ممکن است خطا داشته باشد
except:
    # کدی که در صورت بروز خطا اجرا می‌شود
```

```
try:
    result = 10 / 0
except:
    print("خطایی رخ داد")
```

ساختار پایه try-except

آموزشگاه

کامپیوتر

مثال:

راهکار

بلوک‌های کامل try-except

```
try:
    # کد اصلی
except [ExceptionType]:
    # مدیریت خطا
else:
    # اگر خطایی نبود
finally:
    # همیشه اجرا می‌شود
```

پایتون از چهار بلوک اصلی برای مدیریت خطاها استفاده می‌کند:

Try: کدی که می‌خواهیم بررسی کنیم

Except: مدیریت خطاها

Else: اجرا می‌شود اگر خطایی رخ ندهد

Finally: همیشه اجرا می‌شود (چه خطا باشد چه نباشد)

آموزشگاه

کامپیوتر

مثال:

راهکار

```
try:
    num = int(input("عدد وارد کنید: "))
    result = 100 / num
except ValueError:
    print("لطفاً یک عدد معتبر وارد کنید")
except ZeroDivisionError:
    print("خطا: تقسیم بر صفر")
except Exception as e:
    print(f"خطای ناشناخته: {e}")
else:
    print(f"نتیجه: {result}")
finally:
    print("پردازش کامل شد")
```

کار با فایل‌ها در پایتون (open)

در پایتون، برای کار با فایل‌ها از تابع `open()` استفاده می‌کنیم. این تابع امکان خواندن، نوشتن و مدیریت فایل‌ها را فراهم می‌کند.

```
file_object = open(filename, mode)
```

تابع `open()` ساختار پایه

آموزشگاه

حالت‌های (mode) باز کردن فایل

کامپیوتر

راهکار

حالت	توضیح
'r'	خواندن (پیش‌فرض)
'w'	نوشتن (پاک کردن محتوای قبلی)
'a'	الحاق (اضافه کردن به انتهای فایل)
'x'	ایجاد فایل جدید (خطا اگر فایل وجود داشته باشد)
'b'	حالت باینری
't'	حالت متنی (پیش‌فرض)
'+'	خواندن و نوشتن

آموزشگاه

کامپیوتر

راهکار

آموزشگاه

روش‌های باز کردن و خواندن فایل

کامپیوتر

راهکار

1. روش پایه (نیاز به بستن دستی فایل)

```
file = open('example.txt', 'r')
content = file.read()
file.close()
```

2. استفاده از `with` (توصیه شده - بستن خودکار فایل)

```
with open('example.txt', 'r') as file:
    content = file.read()
```

متدهای خواندن فایل

متد	توضیح
<code>.read()</code>	خواندن تمام محتوای فایل
<code>.read(size)</code>	خواندن مقدار مشخصی از فایل
<code>.readline()</code>	خواندن یک خط
<code>.readlines()</code>	خواندن تمام خطوط به صورت لیست

آموزشگاه

کامپیوتر

راهکار

```
with open('example.txt', 'r') as file:
    # خواندن کل فایل
    print(file.read())

    # برگشت به ابتدای فایل
    file.seek(0)

    # خواندن خط به خط
    for line in file:
        print(line.strip())
```

مثال :

```
with open('output.txt', 'w') as file:
    file.write('Hello World!\n')
    file.write('This is a new line.')
```

```
with open('output.txt', 'a') as file:
    file.write('\nThis is appended text.')
```

نوشتن در فایل

آموزشگاه

1. نوشتن فایل جدید ('w')

کامپیوتر

2. الحاق به فایل ('a')

راهکار

نکات مهم

- همیشه از `with` استفاده کنید تا فایل به صورت خودکار بسته شود.
- برای فایل‌های بزرگ، از خواندن کل فایل خودداری کنید و به جای آن خط به خط پردازش کنید.
- هنگام نوشتن در فایل، حتماً کاراکتر `\n` را برای خط جدید اضافه کنید.
- برای کار با فرمت‌های خاص (CSV, JSON, etc.) از ماژول‌های اختصاصی پایتون استفاده کنید.

کار با فایل‌های CSV در پایتون

فایل‌های CSV (مخفف Comma-Separated Values) یکی از رایج‌ترین فرمت‌ها برای ذخیره و تبادل داده‌های جدولی هستند. پایتون ماژول‌های قدرتمندی برای کار با این نوع فایل‌ها ارائه می‌دهد.

روش‌های کار با CSV در پایتون

(1) استفاده از ماژول CSV (استاندارد)

ماژول CSV که بخشی از کتابخانه استاندارد پایتون است، امکانات کاملی برای خواندن و نوشتن فایل‌های CSV ارائه می‌دهد. مثال خواندن CSV:

```
import csv

with open('data.csv', 'r', encoding='utf-8') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)  # یک لیست از مقادیر است row هر
```

آموزشگاه

مثال نوشتن CSV:

```
import csv

data = [
    ['نام', 'سن', 'شهر'],
    ['رضا', '30', 'تهران'],
    ['سارا', '25', 'مشهد']
]

with open('output.csv', 'w', encoding='utf-8', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

نکات مهم راهکار

- همیشه encoding مناسب مثلاً (utf-8) را مشخص کنید.
- برای جلوگیری از خطوط خالی بین رکوردها، پارامتر `newline=""` را تنظیم کنید.
- برای فایل‌های بزرگ، از خواندن تمام محتوا در حافظه خودداری کنید.
- در صورت امکان از کتابخانه `pandas` برای پردازش داده‌های پیچیده استفاده کنید.
- برای داده‌های حساس، از `quoting` مناسب استفاده کنید تا مشکلات فرمت‌بندی نداشته باشید.

ایجاد رابط گرافیکی در پایتون

مقایسه Kivy و Tkinter برای توسعه رابط کاربری در پایتون

ویژگی‌های کلیدی Tkinter:

- کتابخانه استاندارد پایتون (نیاز به نصب جداگانه ندارد)
- سبک و ساده برای برنامه‌های دسکتاپ
- ظاهر بومی (Native) سیستم عامل
- مناسب برای برنامه‌های ساده تا متوسط

مثال ساده:

```
import tkinter as tk

root = tk.Tk()
root.title("نمونه Tkinter")

label = tk.Label(root, text="!سلام دنیا")
label.pack()

button = tk.Button(root, text="کلیک کن", command=lambda: print("کلیک شد"))
button.pack()

root.mainloop()
```

نقاط قوت Tkinter:

- یادگیری آسان
- حجم کم و اجرای سریع
- مستندات گسترده
- مناسب برای برنامه‌های کنسولی که نیاز به رابط گرافیکی ساده دارند

محدودیت‌ها در Tkinter:

- طراحی‌های پیچیده سخت‌تر است
- ظاهر قدیمی‌تر
- پشتیبانی محدود از افکت‌های پیشرفته

ویژگی‌های کلیدی kivi:

- کتابخانه چندسکویی (دسکتاپ، موبایل، تبلت)
- مبتنی بر OpenGL برای گرافیک پیشرفته
- از لمس و حرکات چندگانه پشتیبانی می‌کند
- مناسب برای برنامه‌های تعاملی و مدرن

مثال ساده:

```
from kivy.app import App
from kivy.uix.button import Button

class MyApp(App):
    def build(self):
        return Button(text='!سلام دنیا',
                      on_press=lambda x: print("!کلیک شد"))

MyApp().run()
```

نقاط قوت kivi:

- رابط کاربری مدرن و قابل تنظیم
- پشتیبانی از چندپلتفرمی (ویندوز، لینوکس، مک، اندروید، iOS)
- مناسب برای برنامه‌های لمسی
- قابلیت ساخت رابط‌های پیچیده با انیمیشن

محدودیت‌ها در kivi:

- یادگیری پیچیده‌تر
- حجم بیشتر برنامه
- ظاهر غیربومی (نیاز به طراحی اختصاصی برای هر پلتفرم)

مدیریت رابط گرافیکی در tkinter

Tkinter کتابخانه استاندارد پایتون برای ایجاد رابطهای گرافیکی (GUI) است. در این آموزش به صورت کامل با Tkinter و ویجت‌های اصلی آن آشنا می‌شوید.

```
import tkinter as tk

root = tk.Tk()
root.title("برنامه من")
root.geometry("400x300") # ارتفاع x عرض
root.mainloop()
```

ساخت پنجره اصلی
آموزشگاه

کامپیوتر

راهکار

ویجت‌های اصلی Tkinter

(1) Label یا برچسب:

```
label = tk.Label(root, text="!سلام دنیا", font=("Arial", 16), fg="blue")
label.pack()
```

کامپیوتر

(2) Button یا دکمه:

```
def button_click():
    print("!دکمه کلیک شد")

button = tk.Button(root, text="کلیک کن", command=button_click, bg="yellow")
button.pack()
```

آموزشگاه

(3) Entry یا ورودی متن:

```
entry = tk.Entry(root, width=30)
entry.pack()
```

```
# برای خواندن مقدار
value = entry.get()
```

کامپیوتر

راهکار

```
text_box = tk.Text(root, height=5, width=30)
text_box.pack()
```

(4) Text یا جعبه متن چند خطی:

```
# درج متن
text_box.insert(tk.END, "\nمتن اولیه")

# خواندن محتوا
content = text_box.get("1.0", tk.END)
```

checkbox(5) یا چک باکس:

```

check_var = tk.IntVar()
check = tk.Checkbutton(root, text="موافقم", variable=check_var)
check.pack()

# بررسی وضعیت
if check_var.get() == 1:
    print("انتخاب شده")

```

radiobutton (6) یا دکمه رادیویی:

```

radio_var = tk.StringVar()
radio1 = tk.Radiobutton(root, text="گزینه ۱", variable=radio_var, value="1")
radio2 = tk.Radiobutton(root, text="گزینه ۲", variable=radio_var, value="2")
radio1.pack()
radio2.pack()

# دریافت مقدار انتخاب شده
selected = radio_var.get()

```

Listbox (7) یا لیست باکس:

```

listbox = tk.Listbox(root, height=4)
for item in ["آیتم ۱", "آیتم ۲", "آیتم ۳"]:
    listbox.insert(tk.END, item)
listbox.pack()

# دریافت آیتم انتخاب شده
selected = listbox.get(tk.ACTIVE)

```

combobox (8) یا کامبویباکس:

```

from tkinter import ttk

combo = ttk.Combobox(root, values=["گزینه ۱", "گزینه ۲", "گزینه ۳"])
combo.pack()

# تنظیم مقدار پیش‌فرض
combo.current(0)

# دریافت مقدار انتخاب شده
selected = combo.get()

```

(9) scrollbar یا نوار پیمایش:

```

scrollbar = tk.Scrollbar(root)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

text = tk.Text(root, yscrollcommand=scrollbar.set)
text.pack()

scrollbar.config(command=text.yview)

```

آموزشگاه

(10) frame یا قاب:

```

frame = tk.Frame(root, borderwidth=2, relief="groove")
frame.pack(padx=10, pady=10)

# اضافه کردن ویجت به فریم
tk.Label(frame, text="این داخل فریم است").pack()

```

راهکار

آموزشگاه

کامپیوتر

راهکار

آموزشگاه

کامپیوتر

راهکار

مدیریت چیدمان و یجتها روی پنجره (Geometry Management)

مدیریت چیدمان در Tkinter یکی از اساسی ترین مفاهیم برای طراحی رابطهای کاربری است. Tkinter سه روش اصلی برای چیدمان و یجتها ارائه می دهد:

1) روش pack()

ساده ترین روش برای چیدمان و یجتها که به صورت خودکار آنها را در پنجره قرار می دهد.

پارامترهای کلیدی:

- Side: تعیین جهت چیدمان (TOP, BOTTOM, LEFT, RIGHT)
- Fill: پر کردن فضای موجود (X, Y, BOTH, NONE)
- Expand: گسترش و یجت هنگام تغییر اندازه پنجره (True/False)
- padx/pady: فاصله از حاشیه (پیکسل)
- ipadx/ipady: فاصله داخلی (پیکسل)

```
import tkinter as tk

root = tk.Tk()

# چیدمان عمودی (پیش فرض)
label1 = tk.Label(root, text="برچسب ۱", bg="red")
label1.pack(fill=tk.X, padx=5, pady=5)

label2 = tk.Label(root, text="برچسب ۲", bg="green")
label2.pack(fill=tk.X, padx=5, pady=5)

# چیدمان افقی
frame = tk.Frame(root)
button1 = tk.Button(frame, text="دکمه ۱")
button1.pack(side=tk.LEFT, padx=5)
button2 = tk.Button(frame, text="دکمه ۲")
button2.pack(side=tk.LEFT, padx=5)
frame.pack(pady=10)

root.mainloop()
```

مزایا:

- ساده و سریع برای چیدمانهای پایه
- نیاز به محاسبات دستی کم

معایب: آموزشگاه

- کنترل دقیق روی موقعیت و یجتها محدود است
- برای رابطهای پیچیده مناسب نیست

(2) روش grid():

چیدمان شبکه‌ای که ویجت‌ها را در ردیف‌ها و ستون‌ها سازماندهی می‌کند.

پارامترهای کلیدی:

- row/column: موقعیت سلول
- rowspan/columnspan: گسترش روی چند سلول
- sticky: چسبیدن به جهت‌های خاص (N, S, E, W, NE, etc.)
- padx/pady: فاصله از حاشیه
- ipadx/ipady: فاصله داخلی

مثال:

```
import tkinter as tk

root = tk.Tk()

# ایجاد فرم ورود اطلاعات
tk.Label(root, text="نام کاربری:").grid(row=0, column=0, sticky=tk.E, padx=5, pady=5)
tk.Entry(root).grid(row=0, column=1, sticky=tk.W, pady=5)

tk.Label(root, text="رمز عبور:").grid(row=1, column=0, sticky=tk.E, padx=5, pady=5)
tk.Entry(root, show="*").grid(row=1, column=1, sticky=tk.W, pady=5)

# دکمه‌ها در یک ردیف با گسترش روی دو ستون
frame = tk.Frame(root)
frame.grid(row=2, columnspan=2, pady=10)
tk.Button(frame, text="ورود").pack(side=tk.LEFT, padx=5)
tk.Button(frame, text="انصراف").pack(side=tk.LEFT, padx=5)

# تنظیم اندازه ستون‌ها
root.columnconfigure(1, weight=1)

root.mainloop()
```

مزایا:

- کنترل دقیق روی موقعیت ویجت‌ها
- مناسب برای فرم‌ها و چیدمان‌های جدولی
- امکان ترکیب با pack() در فریم‌های مختلف

معایب:

- نیاز به برنامه‌ریزی دقیق شبکه
- پیچیدگی بیشتر نسبت به pack()

(3) روش place:

قرار دادن ویجت‌ها با مختصات دقیق (مطلق یا نسبی)

پارامترهای کلیدی:

- x/y: موقعیت مطلق (پیکسل)
- relx/rely: موقعیت نسبی (0.0 تا 1.0)
- width/height: اندازه مطلق
- relwidth/relheight: اندازه نسبی
- anchor: نقطه اتکا (N, S, E, W, CENTER, etc.)

```
import tkinter as tk

root = tk.Tk()
root.geometry("400x300")

# دکمه با موقعیت مطلق
button1 = tk.Button(root, text="دکمه ۱")
button1.place(x=50, y=30, width=100, height=40)

# دکمه با موقعیت نسبی
button2 = tk.Button(root, text="دکمه ۲")
button2.place(relx=0.5, rely=0.5, anchor=tk.CENTER)

# ویجت با اندازه نسبی
frame = tk.Frame(root, bg="blue")
frame.place(relx=0.1, rely=0.1, relwidth=0.8, relheight=0.8)

root.mainloop()
```

مزایا:

- کنترل کامل روی موقعیت و اندازه
- مناسب برای رابط‌های سفارشی و پیچیده
- امکان طراحی واکنش‌گرا (Responsive)

معایب:

- نیاز به محاسبات دستی
- عدم انعطاف در تغییر اندازه پنجره
- نگهداری و تغییر سخت‌تر

نکات پیشرفته tkinter

```
from tkinter import ttk
button = ttk.Button(root, text="دکمه مدرن")
```

استفاده از ttk برای ظاهر مدرن تر :

آموزشگاه

تغییر تم (theme):

```
style = ttk.Style()
style.theme_use("clam") # سایر تم ها: 'alt', 'default', 'classic'
```

راهکار

کار با تصاویر:

```
photo = tk.PhotoImage(file="image.png")
label = tk.Label(root, image=photo)
```

آموزشگاه

```
def open_new_window():
    new_window = tk.Toplevel(root)
    tk.Label(new_window, text="پنجره جدید").pack()
```

چند پنجره‌ای:

راهکار

آموزشگاه

کامپیوتر

راهکار

کار با SQLite3 در پایتون

SQLite3 یک پایگاه داده سبک و تعبیه شده است که در پایتون به صورت داخلی پشتیبانی می‌شود. در اینجا مراحل و مفاهیم اصلی کار با SQLite3 در پایتون را بررسی می‌کنیم:

اتصال به پایگاه داده:

```
import sqlite3

# اتصال به پایگاه داده (اگر وجود نداشته باشد، ایجاد می‌شود)
conn = sqlite3.connect('mydatabase.db')

# برای اجرای دستورات SQL cursor ایجاد یک
cursor = conn.cursor()
```

ایجاد جدول (Create Table):

```
cursor.execute('''CREATE TABLE IF NOT EXISTS users
                (id INTEGER PRIMARY KEY,
                 name TEXT,
                 age INTEGER,
                 email TEXT)''')

conn.commit()
```

افزودن رکورد جدید به جدول (Insert into):

```
# افزودن یک رکورد
cursor.execute("INSERT INTO users (name, age, email) VALUES (?, ?, ?)",
              ('Ali', 30, 'ali@example.com'))
conn.commit()

# افزودن چند رکورد به صورت یکجا
users_data = [
    ('Reza', 25, 'reza@example.com'),
    ('Maryam', 28, 'maryam@example.com'),
    ('Ahmad', 35, 'ahmad@example.com')
]

cursor.executemany("INSERT INTO users (name, age, email) VALUES (?, ?, ?)", users_data)
conn.commit()
```

خواندن داده‌ها از جدول (Select):

```
# خواندن تمام رکوردها
cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()
for row in rows:
    print(row)

# خواندن با شرایط خاص
cursor.execute("SELECT name, email FROM users WHERE age > ?", (25,))
filtered_rows = cursor.fetchall()
for row in filtered_rows:
    print(row)
```

بروزرسانی داده‌ها (Update):

```
cursor.execute("UPDATE users SET age = ? WHERE name = ?", (31, 'Ali'))
conn.commit()
```

حذف داده (Delete):

```
cursor.execute("DELETE FROM users WHERE age < ?", (26,))
conn.commit()
```

```
# بستن cursor و connection
cursor.close()
conn.close()
```

بستن اتصال: آموزشگاه

کامپیوتر

دسترسی به فیلدها با نام معین:

```
conn.row_factory = sqlite3.Row
cursor = conn.cursor()
cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()
for row in rows:
    print(row['name'], row['email']) # دسترسی با نام ستون
```